

11 - Prática: Pipelines de Dados I - Apache Airflow (I)

Thassiana Camilia Amorim Muller

Introdução

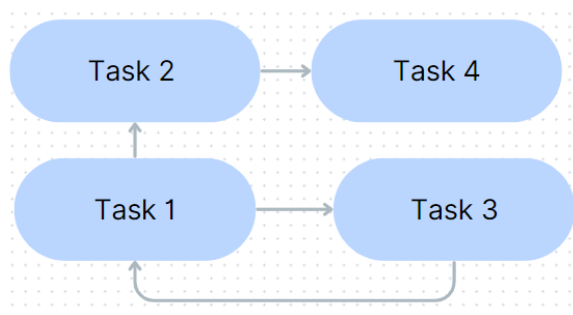
O Apache Airflow é uma ferramenta de código aberto utilizada para automatizar e gerenciar *workflows* para projetos de dados. Desenvolvido originalmente pela Airbnb, ele permite criar, agendar e monitorar *pipelines* de processamento usando Python para definir os workflows como grafos acíclicos direcionados (DAGs).

Possui uma interface de fácil uso auxiliando no acompanhamento das tarefas, é flexível e integra-se com bancos de dados, ferramentas de big data e serviços em nuvem, ideal para tarefas como ETL (Extract, Transform, Load), processamento em lote e orquestração de processos complexos, por essa razão, é muito usado por engenheiros de dados e DevOps.

Grafos acíclicos direcionados - DAGs

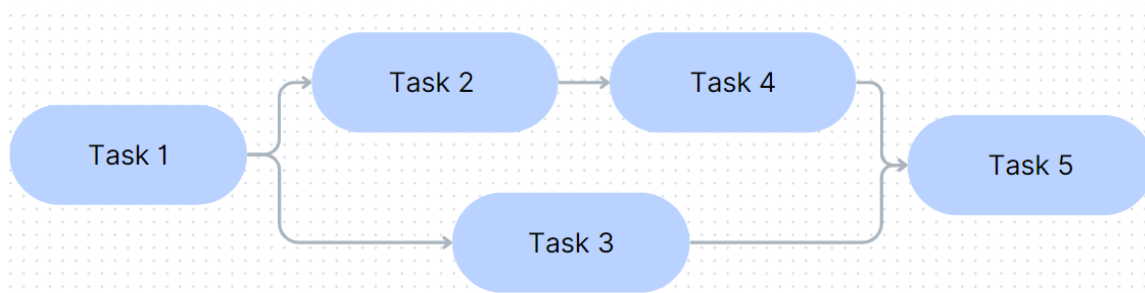
No contexto do Apache Airflow, DAGs são grafos onde cada nó representa uma tarefa. O grafo não pode conter ciclos.

Exemplo de grafo cíclico



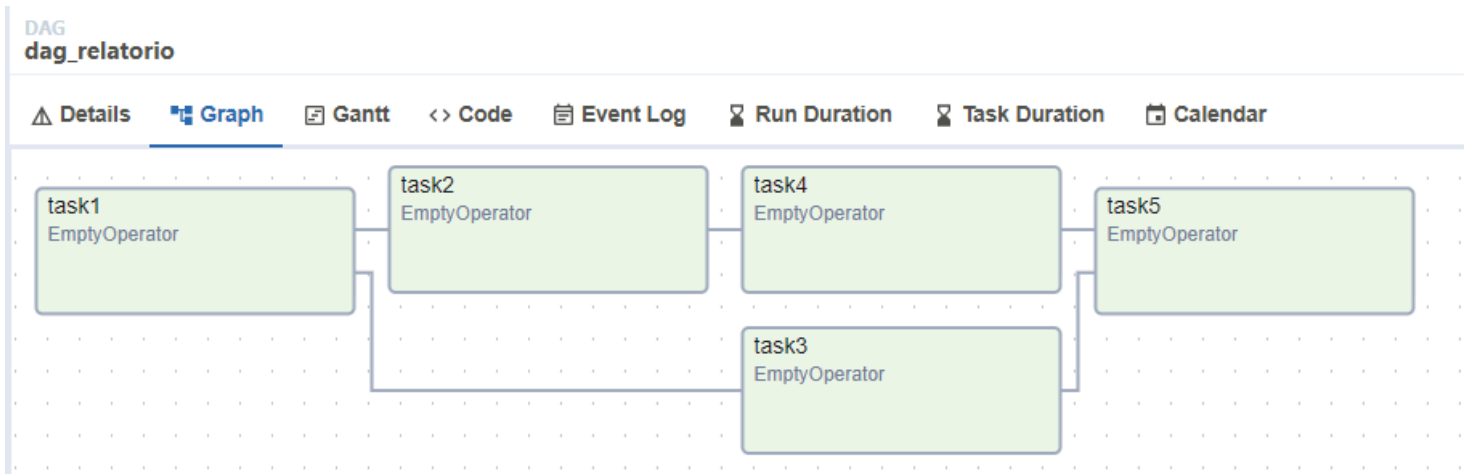
Fonte: Autoria própria

Exemplo de grafo acíclico



Fonte: Autoria própria

Exemplo da DAG acíclica acima no webserver do Airflow



Fonte: Autoria própria

Código Python que representa a DAG acima

```
from airflow import DAG
from airflow.operators.empty import EmptyOperator
from datetime import datetime

default_args = {
    "owner": "airflow",
    "start_date": datetime(2025, 4, 1),
}

with DAG(
    dag_id="dag_relatorio",
    default_args=default_args,
    schedule=None,
    catchup=False,
) as dag:

    task1 = EmptyOperator(task_id="task1")
    task2 = EmptyOperator(task_id="task2")
    task3 = EmptyOperator(task_id="task3")
    task4 = EmptyOperator(task_id="task4")
    task5 = EmptyOperator(task_id="task5")

    # task1 roda primeiro
    task1 >> [task2, task3]
```

```
# task4 depende da task2
task2 >> task4

# task5 depende da task3 e da task4
[task3, task4] >> task5
```

As tarefas por si são criadas de maneira independente e suas dependências são explicitadas na última linha do código.

Aplicações

- **ETL/ELT:** Automatizar pipelines de extração, transformação e carga de dados.
- **Machine Learning:** Orquestrar treinamento de modelos e processamento de dados.
- **Processamento em Lote:** Executar tarefas agendadas, como relatórios e atualizações de dados.
- **Integração entre Sistemas:** Coordenar processos entre diferentes serviços e APIs.

Arquitetura

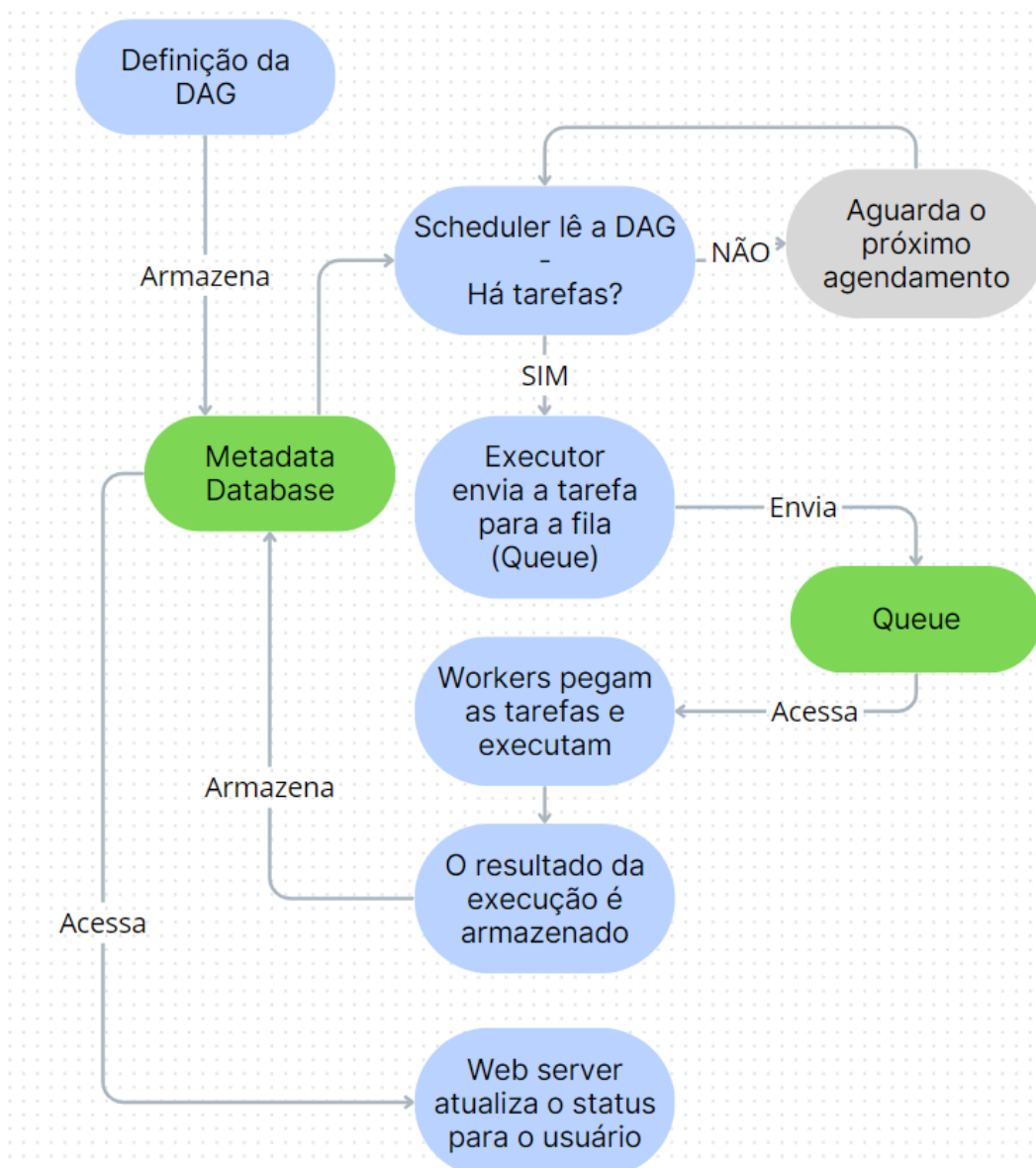
O Apache Airflow possui uma arquitetura modular e distribuída, projetada para orquestrar workflows representados por DAGs de maneira escalável e confiável. Sua estrutura é composta por componentes que trabalham em conjunto para agendar, executar e monitorar tarefas.

Esses componentes são:

- **Web Server:** interface gráfica que permite visualizar DAGs, logs, status de execução e gerenciar tarefas manualmente.
- **Executor:** define como e onde as tarefas são executadas. Exemplos de executor:
 - **SequentialExecutor:** padrão, executa tarefas sequencialmente, apenas para testes.
 - **LocalExecutor:** executa tarefas em paralelo na mesma máquina.
 - **CeleryExecutor:** distribui tarefas em workers usando Celery + Redis/RabbitMQ.
 - **KubernetesExecutor:** executa cada tarefa em um pod Kubernetes.

- **Scheduler:** responsável por analisar as DAGs e agendar tarefas, enviar tarefas para o Executor, monitorar o estado das tarefas e reagir a falhas ou retries.
- **Metadata Database:** armazena estados, configurações, DAGs, logs e históricos de execução.
- **DAG Folder:** pasta onde os arquivos DAGs são armazenados (geralmente em Python).
- **Workers:** processos que executam as tarefas, no caso do **CeleryExecutor** ou **KubernetesExecutor**

Fluxo de funcionamento:



Fonte: Autoria própria

1. O **Scheduler** lê os DAGs do **DAG Folder** e os armazena no **Metadata Database**.
2. **Agendamento**: o Scheduler verifica quais tarefas devem ser executadas com base em seus agendamentos (**schedule_interval**) definidos no DAG.
3. **Enfileiramento**: o **Scheduler** envia tarefas prontas para o **Executor**, que as coloca em uma fila (se for CeleryExecutor).
4. **Execução**: workers pegam tarefas da fila e as executam.
5. **Atualização de Status**: o resultado da execução é registrado no Metadata Database.
6. **Monitoramento**: a Web Server exibe o status em tempo real na interface.

Modos de instalação

- **Standalone**: todos os componentes (Web Server, Scheduler, Executor) rodam em um único servidor. Usado para desenvolvimento ou ambientes pequenos.
- **Celery + Redis/RabbitMQ**: sistema distribuído, workers rodam em máquinas separadas. Escalável horizontalmente se adicionar mais workers aumenta a capacidade.
- **Kubernetes (Airflow 2.0+)**: cada tarefa é executada em um Pod Kubernetes efêmero. Alta elasticidade e isolamento entre tarefas.

Operators

Define uma unidade de trabalho dentro da DAG. Cada operator representa uma task dentro do fluxo de trabalho e pode ser reutilizado em diferentes DAGs.

Exemplos de operadores de ação

- **BashOperator**: executa comandos em shell.
- **PythonOperator**: executa funções Python.
- **EmailOperator**: envia e-mails automaticamente.

Exemplos de operadores de transferência:

- **PrestoToMySQLOperator**: move dados entre sistemas (não recomendado para grandes quantidades de dados).

Exemplos de operadores de transferência:

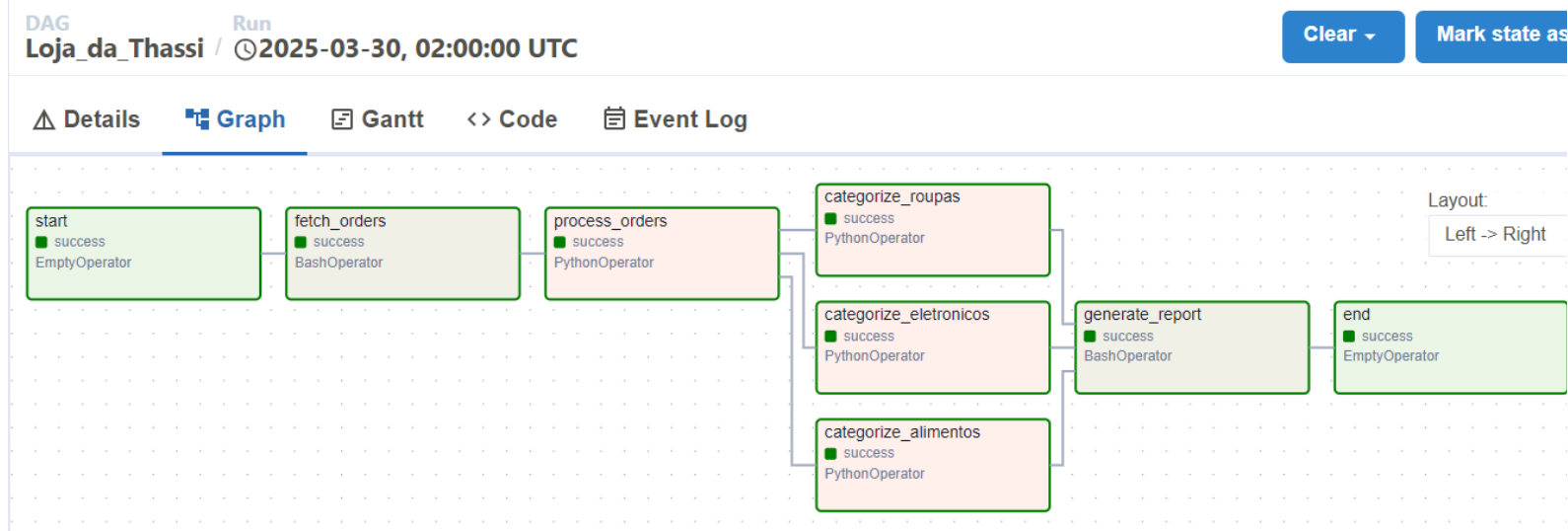
- **FileSensor:** verifica estados.

Aplicação Prática - Loja da Thassi

Para aplicar os conhecimentos adquiridos no curso, utilizei o Airflow para criar um fluxo de simulação de um e-commerce.

Primeiramente, criei uma DAG com o fuso horário de São Paulo que fica programada para rodar todo dia às 2h da manhã a partir de 29 de março de 2025, com a configuração de realizar novas tentativas em caso de falha.

Após configurar a DAG, crio uma task vazia (start) para marcar o início da execução, seguida pela task “fetch_orders” que simula baixar os pedidos feitos na loja com um comando bash. Após o término da task de baixar os pedidos, é iniciada a task que simula o processamento dos pedidos “process_orders_task” através de uma função Python, e em seguida roda a task do processamento específico por categorias (eletrônicos, roupas e alimentos) “categorize_tasks”. Após o processamento das categorias, um relatório final é gerado por uma task via comando bash, e o fluxo é finalizado com uma task vazia (end).



Fonte: Autoria própria

O fluxo pode ser resumido em:

1. Procura dos pedidos

- Simulado por BashOperator que faz um print no terminal (fetch_orders)

2. Processamento dos pedidos

- Simulado por um PythonOperator que chama `process_orders()`.

3. Separação dos pedidos em categorias

- Criação dinâmica de tarefas para processar pedidos de **eletrônicos, roupas e alimentos**.
- Cada categoria tem sua própria task gerada dinamicamente no loop `categories`.

4. Geração de relatório (`generate_report`)

- Outro BashOperator que simula a geração de um relatório final.

5. Fim do fluxo (`end`)

- Um EmptyOperator para marcar o fim do pipeline.

Conclusão

Assim, conclui-se que o Airflow é uma ferramenta muito útil para gerenciar, agendar e monitorar fluxos de forma eficiente, com uma curva de aprendizagem simples e uma interface gráfica intuitiva.

Referências

Agrupar tarefas dentro de DAGs. Disponível em: [Agrupar tarefas dentro de DAGs | Cloud Composer](#) Acesso em: 25 mar. 2025.

ANDRÉ RICARDO. O que é o Apache Airflow? Disponível em:

 O que é o Apache Airflow ? Acesso em: 25 mar. 2025.

INZAUGARAT, M. E. What is a DAG? A Practical Guide with Examples. Disponível em: [What is a DAG? A Practical Guide with Examples | DataCamp](#) Acesso em: 25 mar. 2025.

